

Senior QA Automation Engineer Interview Cheatsheet

Selenium & Test Automation (Java Focus)

1. How would you design a scalable Selenium automation framework in Java?

Answer: Use layered architecture: test layer (JUnit/TestNG), Page Object Model for maintainability, utility layer for driver setup/waits, configuration files for environments, reporting integration (Allure/Extent), CI/CD integration with Jenkins or GitHub Actions.

2. What is the difference between implicit, explicit, and fluent waits?

Answer: Implicit sets a global timeout for element lookups. Explicit waits for a specific condition on an element. Fluent adds polling and exception handling. Best practice is explicit waits.

3. How do you handle dynamic elements in Selenium?

Answer: Use flexible locators (XPath with contains/starts-with), explicit waits, stable attributes (data-test-id).

Code Example:

```
WebElement btn = new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.xpath("//button[contains(text(),'Submit')]")));
btn.click();
```

4. What patterns improve maintainability in automation frameworks?

Answer: Page Object Model, Page Factory, Screenplay Pattern, Data-Driven Tests, Driver Factory, Decorator Pattern.

5. How would you manage test data in automation?

Answer: Use externalized files (JSON/YAML/Excel), APIs/DB for setup and teardown, unique data with UUIDs, environment config outside code.

6. How do you handle cross-browser and parallel test execution?

Answer: TestNG parallel runs, Selenium Grid, Dockerized Grid, BrowserStack/Sauce Labs, driver factory with profiles.

7. How do you debug flaky Selenium tests?

Answer: Add logs and screenshots, replace sleeps with waits, retry analyzer in TestNG, determine if issue is locator/timing vs environment.

8. How do you integrate Selenium tests into CI/CD pipelines?

Answer: Store in Git with Maven/Gradle, Jenkins/GitHub Actions integration, Docker for browsers, publish reports, run tagged groups for smoke vs regression.

9. How do you use Java streams and collections in test automation?

Answer: Cleaner code and assertions.

```
List productNames = driver.findElements(By.cssSelector(".product-title"))
    .stream()
    .map(WebElement::getText)
    .collect(Collectors.toList());
```

10. How would you test an application with heavy AJAX usage?

Answer: Explicit waits for conditions, wait for Ajax with JavaScript.

```
new WebDriverWait(driver, Duration.ofSeconds(15)).until(
    wd -> ((JavascriptExecutor wd).executeScript("return jQuery.active == 0")).equals(true));
```

11. How do you approach API + UI integrated testing?

Answer: Use APIs for setup/validation, UI for workflows, combine Selenium with RestAssured/HttpClient.

12. What are Selenium limitations and how do you overcome them?

Answer: Cannot handle CAPTCHAs (bypass in test env), limited desktop support (Autolt/WinAppDriver), file handling via Robot/ChromeOptions, slower execution (combine with API tests).

SQL & Database Testing

1. How do you validate backend data after UI actions?

Answer: Use JDBC in Java to connect and run queries. Compare DB state to expected.

```
Connection conn = DriverManager.getConnection(url, user, pass);
```

```
Statement stmt = conn.createStatement();
```

```
ResultSet rs = stmt.executeQuery("SELECT status FROM orders WHERE order_id=123");
```

```
rs.next();
```

```
Assert.assertEquals(rs.getString("status"), "CONFIRMED");
```

2. What SQL queries are common in QA automation?

Answer: SELECT for validations, INSERT for test data setup, UPDATE for state changes, DELETE for cleanup. Use joins for cross-table verification.

3. How do you handle test data setup and cleanup with databases?

Answer: Pre-seed test data with INSERT or via APIs. Roll back transactions after tests, or truncate/reset tables. Use flyway/liquibase migrations for versioning.

4. How do you ensure database integrity in tests?

Answer: Verify foreign keys, unique constraints, check triggers. After test actions, query related tables to confirm cascade or rollback.

5. How do you optimize database checks in automation?

Answer: Minimize queries inside tests, group verifications in one query, avoid large data scans, index columns used in lookups.

6. What are common SQL performance checks a QA engineer might do?

Answer: Detect missing indexes, check query execution plans, measure response times, validate that queries scale with data volume.

7. How do you handle database differences across environments?

Answer: Keep queries generic (ANSI SQL), externalize DB configs, use property files, apply migrations to keep schemas consistent.

8. How would you combine SQL checks with Selenium tests?

Answer: UI triggers an action (e.g., order placed). After action, validate confirmation in UI and confirm DB state with SQL query.

9. Example Senior-Level SQL Query Question

Question: Write a query to find customers who placed more than 5 orders last month.

```
SELECT customer_id, COUNT(*) as total_orders
```

```
FROM orders
```

```
WHERE order_date >= DATEADD(MONTH, -1, GETDATE())
```

```
GROUP BY customer_id  
HAVING COUNT(*) > 5;
```

10. How would you secure DB credentials in automation?

Answer: Use environment variables, encrypted secrets in CI/CD, never hardcode in tests.